

RELAYSPEC: ADAPTING FROZEN SPECULATIVE DRAFTERS WITH LINEAR MAPS

Anonymous authors

Paper under double-blind review

ABSTRACT

As Large Language Models (LLMs) continue to advance, efficient inference remains a challenge. Speculative decoding reduces sequential generation by drafting several tokens for parallel target verification. Drafters such as EAGLE and DFlash use target hidden states, making reuse harder when the target changes. We introduce RelaySpec, which adapts a pretrained DFlash drafter by training five linear maps on its target-feature inputs while keeping the draft transformer frozen. For fine-tuned targets, we train the maps to improve token proposals using target-generated responses. For cross-model transfer, we train them to reconstruct the features expected by the drafter. The maps fold into the existing fusion projection before inference. The same construction applies to EAGLE-3 with three maps, which gain 46 to 106 percent on fine-tuned targets. We evaluate adaptation to fine-tuned targets, transfer across model sizes and families, and shared serving. A Qwen3-4B drafter transferred to Qwen3-8B reaches $5.82\times$ autoregressive throughput on MATH and $4.68\times$ on GSM8K, recovering 100.2% and 99.4% of native-drafter throughput. Fitting these maps uses 16,384 training examples, about 49 times fewer than the published recipe for training a DFlash drafter, and updates no draft weight. Transfer is weaker on code and chat. A nonlinear map with more parameters does not improve on the linear one in our benchmarks. Comparisons with draft-body adaptation and feature interventions show when adapting the drafter’s inputs is useful and where its benefits are limited.

1 INTRODUCTION

As Large Language Models (LLMs) continue to advance, efficient inference remains a challenge. Autoregressive decoding generates one token at a time. This inherent seriality creates a major performance bottleneck: decoding is slow, memory-bound, and fails to fully utilize modern GPUs (Cai et al., 2024).

Speculative decoding accelerates a language model, called the *target*, using a smaller model or prediction module, called the *drafter*. The drafter proposes several candidate tokens, and the target checks them in parallel. Verification keeps an accepted prefix and replaces the first rejected proposal with a correction. Appropriate acceptance and correction rules preserve the target’s output distribution (Leviathan et al., 2023; Chen et al., 2023). Speedup depends on how many proposals are accepted and the time spent drafting and verifying them.

Some drafters also use the target’s hidden states, the internal vectors it computes while processing the preceding text. These vectors provide information about the context that can help the drafter propose tokens the target will accept. Medusa adds parallel prediction heads to the target’s final hidden states (Cai et al., 2024). The EAGLE family uses target features to guide autoregressive drafting (Li et al., 2024b;c; 2025). DFlash instead uses a block-diffusion drafter to predict several tokens in one pass, with target hidden states providing context to each draft layer (Chen et al., 2026).

This dependence complicates reuse when a service changes its target’s size or family, or fine-tunes it using methods such as LoRA (Hu et al., 2022). The new target can produce different hidden states and preferred continuations, and changed feature dimensions may be incompatible with the existing drafter. Training a replacement requires target-specific supervision, draft-weight optimization, and validation (Li et al., 2025; Chen et al., 2026). Repeating this process across target variants increases

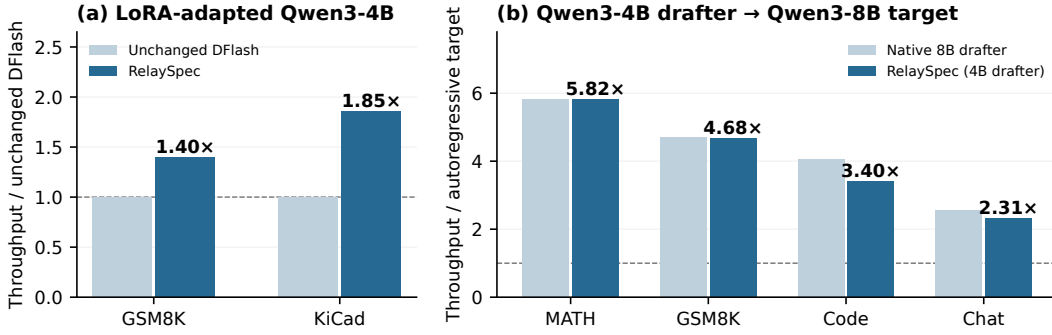


Figure 1: RelaySpec adapts a frozen drafter to a changed target. (a) On LoRA-adapted Qwen3-4B targets, the maps raise mean per-request throughput by 39.51% on GSM8K and 85.47% on KiCad over the unchanged DFlash drafter. (b) A Qwen3-4B drafter reused with Qwen3-8B reaches $5.82\times$ and $4.68\times$ autoregressive throughput on MATH and GSM8K, with the native 8B drafter shown for comparison. The panels use different baselines. Fitting the maps took under six minutes in (a) and 25 minutes in (b), and no draft weight is updated. Section 5.1 gives the evaluation settings.

training and maintenance costs. Can we instead reuse an existing drafter by adapting only its target-feature inputs?

We introduce RelaySpec, a method for reusing a pretrained DFlash drafter after its target changes. Section 4 describes the architecture. The new target supplies hidden states from five layers, as defined in Section 2.1. A separate linear map transforms each stream before the drafter’s original fusion projection combines them. The resulting context conditions the draft transformer, which predicts a block of candidate tokens. The new target verifies this block and accepts or corrects the proposals through the existing decoding procedure.

During adaptation, only the five maps are trained. The target, original fusion projection, draft transformer, normalization layers, and the embeddings and output head selected for the model pair remain frozen. A map can be rectangular when the target and drafter expect different feature dimensions. Before inference, we multiply each map into its corresponding fusion block, a composition of linear operations. This produces one effective projection, so inference needs no separate mapping operation. Because only the maps are trained, adaptation needs far less data than building a drafter. The published DFlash recipe trains its draft transformer on a mixture of about 800,000 samples (Chen et al., 2026). Our selected maps use 4,096 examples for a LoRA-adapted target and 16,384 for cross-model transfer, about 195 and 49 times fewer.

For a LoRA-fine-tuned target, identity initialization leaves its hidden states unchanged and preserves the existing drafter’s behavior with that target. The maps therefore introduce no additional disruption at initialization. Training then modifies those hidden states so that they better condition the frozen drafter to predict the LoRA-adapted target’s tokens. We use *accept-until-fail (AUF)*: cross-entropy on saved target responses supervises the correct draft prefix and its first incorrect proposal (Yang & Li, 2026). Later positions are excluded because verification would stop at the earlier error.

For cross-model transfer, the drafter was trained to process hidden states from its original target. Replacing that target changes the features, and sometimes their dimensions, so the same drafter may no longer interpret them effectively. We run the original and new targets on the same text and train linear maps to reconstruct the original target’s features from the new target’s features. The *mean squared error (MSE)* objective has two terms: one compares each mapped feature with its original-target counterpart, and the other compares their combined context after the frozen fusion projection and RMSNorm. The context term supervises what the draft layers actually receive, beyond matching each layer separately. Both terms use squared errors scaled by the corresponding original-target vector’s squared norm, with numerical stabilization.

We evaluate specialization, cross-model transfer, training budgets, and shared serving, and we repeat both adaptation settings on EAGLE-3. Matched comparisons include draft-body updates, direct fusion updates, nonlinear maps, and different token objectives. Their results delimit the method: body adaptation or decaying cross-entropy can be stronger on some workloads, and a pooled adapter

can approach separate task adapters. We therefore measure both single-request and shared-serving performance rather than assume that every target needs a separate drafter or that higher acceptance always gives higher throughput.

We analyze the learned maps to understand why adaptation helps. First, we use singular value decomposition (SVD) to retain only the largest components of either the full map W_i or its correction $\Delta_i = W_i - I$. For the tested GSM8K and NanoCoder checkpoints, truncating the full map to rank one or two sharply reduces acceptance. Using $I + [\Delta_i]_r$, where $[\Delta_i]_r$ is the rank- r truncated correction, preserves more acceptance, though these low ranks do not recover the full-map result. Second, scaling only the correction shows that half strength retains much of the benefit, while stronger corrections reduce acceptance. Third, we compare the correction with the LoRA-induced change in fused hidden states. Their mean cosine similarities are close to zero and similar to shuffled-position controls, providing no evidence of systematic cancellation of the LoRA change. Finally, in cross-size transfer, moving token-trained maps halfway toward an MSE-trained solution improves acceptance and throughput on the tested Math and Code workloads. This supports learning inputs the frozen drafter can use, without establishing a universal advantage for MSE.

Figure 1 summarizes selected throughput results. Our contributions are:

- We propose RelaySpec, which adapts a pretrained DFlash drafter through five linear maps while freezing its transformer. The maps fold into the original fusion projection, and we study token supervision for target specialization and feature reconstruction for cross-model reuse. A nonlinear map with more parameters does not improve on the linear form in our benchmarks.
- We evaluate adaptation across fine-tuned targets, model sizes, and families, including training-budget and shared-serving comparisons. On Qwen3-4B (Yang et al., 2025) LoRA targets, the selected AUF maps improve mean per-request throughput over the unchanged DFlash drafter by 39.51% and 85.47% on GSM8K (Cobbe et al., 2021) and KiCad, respectively. The selected Qwen3-4B-to-8B transfer recovers 100.2% and 99.4% of native-drafter throughput on MATH (Hendrycks et al., 2021) and GSM8K, with weaker transfer on code and chat. Three maps on EAGLE-3 improve on the unadapted drafter by 45.9% to 105.7% on fine-tuned targets and exceed a purpose-built Qwen3-8B EAGLE drafter on all four transfer workloads. The selected maps fit in 2.66 minutes on two L40S GPUs, and Section 5.7 compares that cost with the published drafter training recipe.
- We analyze full-map and correction-map SVD truncation, correction scaling, alignment with LoRA-induced feature changes, and interpolation toward MSE-trained maps.

2 PRELIMINARIES

2.1 TARGETS, DRAFTERS, AND NOTATION

The *original target* is the model used to train the available drafter. The *new target* is the model to be accelerated. A *native drafter* is trained for the target being evaluated. In cross-model transfer, the original target used for feature reconstruction and the native drafter used as a performance baseline are therefore different objects.

DFlash predicts a block of candidate tokens from an anchor token and masked future positions in one forward pass. It fuses selected target hidden states and uses the resulting context to supply keys and values to every draft layer (Chen et al., 2026). A drafter of this kind reads K feature streams, five for our DFlash checkpoints and three for EAGLE-3 (Li et al., 2025). We write the original target’s feature at stream i and position t as $h_{i,t}^{(o)} \in \mathbb{R}^{d_r}$. The original fusion blocks $F_i \in \mathbb{R}^{d_d \times d_r}$ produce

$$c_t^{(o)} = N \left(\sum_{i=1}^K F_i h_{i,t}^{(o)} \right), \quad F = [F_1 \ \cdots \ F_K]. \quad (1)$$

Here d_d is the draft hidden width and N is the original RMSNorm, which normalizes vector magnitude and applies learned coordinate-wise gains (Zhang & Sennrich, 2019). For the new target, $h_{i,t} \in \mathbb{R}^{d_t}$ and $H_i = [h_{i,1} \ \cdots \ h_{i,n}] \in \mathbb{R}^{d_t \times n}$. We use column vectors throughout. A RelaySpec

map has shape $W_i \in \mathbb{R}^{d_r \times d_t}$, so $W_i h_{i,t}$ can enter the original fusion block. The widths d_t , d_r , and d_d need not be equal.

2.2 VERIFICATION AND USEFUL TRAINING POSITIONS

Let $x_{1:t}$ be an accepted prefix. The drafter proposes γ tokens $y_{1:\gamma}$ and the target scores them in parallel under $x_{1:t}, y_{<j}$. With draft distribution q and target distribution p over a shared vocabulary, speculative sampling accepts y_j with probability

$$a_j = \min\left(1, \frac{p(y_j | x_{1:t}, y_{<j})}{q(y_j | x_{1:t}, y_{<j})}\right). \quad (2)$$

At rejection, a correction is sampled from the normalized positive part of $p - q$. If all proposals are accepted, the target provides a further token. Using the actual proposal probabilities preserves the target distribution in exact arithmetic (Leviathan et al., 2023; Chen et al., 2023). Our experiments use greedy decoding: retain the longest prefix matching the target’s greedy choices, then use the target’s correction.

Only a consecutive prefix contributes accepted draft tokens. If $L \in \{0, \dots, \gamma\}$ is its length, then $\mathbb{E}[L] = \sum_{j=1}^{\gamma} \Pr(L \geq j)$. This motivates supervising the first incorrect proposal as well as the correct prefix. AUF applies that rule to comparisons against saved target tokens (Yang & Li, 2026). Its training-time correct prefix is a prediction diagnostic, not a direct measurement of live decoding speed.

3 RELATED WORK

Target-conditioned drafting. Medusa predicts future positions with additional heads on the target’s final hidden states (Cai et al., 2024). EAGLE uses target features for autoregressive drafting, with later versions improving tree construction and training (Li et al., 2024b;c; 2025). ReDrafter combines target-conditioned recurrent drafting with beam search and dynamic tree attention (Cheng et al., 2024). DFlash predicts a masked block in parallel and supplies target context to every draft layer (Chen et al., 2026). RelaySpec starts from a pretrained DFlash checkpoint and studies its reuse after the target changes.

Adapting to changed targets. DistillSpec studies training data and divergence objectives for matching a drafter to its target (Zhou et al., 2024). Online Speculative Decoding updates the drafter from serving feedback (Liu et al., 2024). Efficient Draft Adaptation separates shared and target-specific components and trains the private component using regenerated and selected data (Lin et al., 2026). OmniDraft addresses vocabulary mismatch with an online cache and hybrid distillation (Ramakrishnan et al., 2025). These methods already address parts of the reuse problem. RelaySpec asks how much adaptation can be placed in five maps before a pretrained DFlash fusion projection while retaining the draft transformer.

Parallel drafting across targets. PARD trains an autoregressive model for parallel prediction and supports reuse within a target family (An et al., 2026a). PARD-2 supports both target-conditioned and target-independent drafting (An et al., 2026b). RelaySpec instead starts with a target-conditioned DFlash checkpoint and adapts it to changed targets. Our experiments compare separate and pooled adaptation because target-specific training is not automatically necessary or better.

Steering pretrained drafters. SD² uses linear projections of target hidden states to add steering signals inside a pretrained autoregressive drafter’s feed-forward layers (Berdoz et al., 2026). Its main method trains the steering projections and draft weights jointly. Its frozen-drafter ablation also improves acceptance, although joint training performs better in that comparison. This establishes prior evidence for steering both trainable and frozen drafters. RelaySpec studies input maps that fold into the fusion projection of two drafter families, comparing token supervision for specialization with reconstruction for cross-model transfer.

Linear maps and training-time parameterization. Huang et al. (2025) use linear transformations to transfer concept-steering vectors across language models. Chen et al. (2025) learn affine maps between model activations to transfer features, predictors, and steering vectors. These results motivate mapping hidden states, but do not establish that a mapped drafter retains its acceptance or speed. RepSpec adds redundant linear structures during draft training and merges them for inference (Huo et al., 2026). RelaySpec shares the use of mergeable linear operations, but trains maps on target-feature inputs while preserving the pretrained draft transformer. Our direct-fusion comparisons and matched-weight gradient diagnostics examine how this parameterization changes optimization. They do not attribute a trained checkpoint’s advantage to a single gradient measurement. We also compare against a nonlinear map to test whether a linear interface is sufficient.

4 METHOD

RelaySpec reuses a pretrained speculative drafter when its target is fine-tuned or replaced by a different model. We train linear maps that transform the new target’s hidden states into inputs the existing drafter can use, while keeping the target and the original drafter weights frozen. The drafter uses these mapped features to propose tokens, and the new target verifies them. This lets us adapt the drafter to a changed target without retraining its prediction network. Our main implementation and experiments use DFlash (Chen et al., 2026), so we describe its architecture and training procedures first, then apply the same construction to EAGLE-3.

4.1 ADAPTING THE DRAFTER’S INPUTS

We retain the drafter’s K target-layer inputs and place one map before each original fusion block (Chen et al., 2026). DFlash uses $K = 5$. Map W_i connects the new target’s selected feature stream i to the corresponding original drafter input. The same map is applied at every token position. At position t , let $h_{i,t} \in \mathbb{R}^{d_t}$ be the new target’s feature from selected layer i . A bias-free map $W_i \in \mathbb{R}^{d_r \times d_t}$ converts it to the width d_r expected by the drafter. The frozen fusion block $F_i \in \mathbb{R}^{d_d \times d_r}$ projects it to the draft width d_d . The K streams give the context

$$\hat{c}_t = N \left(\sum_{i=1}^K F_i W_i h_{i,t} \right), \quad (3)$$

where N is the original RMSNorm (Zhang & Sennrich, 2019). Only $W_1, \dots, W_K$ are trained. The target, fusion projection, normalization, and draft transformer remain frozen. We also retain and freeze the original drafter’s token embeddings and vocabulary output head.

After training, we multiply each learned matrix W_i by its corresponding fusion block F_i and join the K products into one matrix:

$$G = [F_1 W_1 \ \cdots \ F_K W_K] \in \mathbb{R}^{d_d \times K d_t}, \quad \hat{c}_t = N(G[h_{1,t}; \dots; h_{K,t}]). \quad (4)$$

The bracketed features are stacked into one vector. Block multiplication gives $G[h_{1,t}; \dots; h_{K,t}] = \sum_i F_i W_i h_{i,t}$, so one multiplication by G replaces the separate map and fusion operations in exact arithmetic. The original normalization remains afterward. Figure 2 shows both computations. We keep the maps linear because a linear map both changes the feature width and folds into the fusion projection. In our benchmarks a nonlinear alternative with more parameters does not improve on it, as Section 5.3.3 reports.

4.2 TRAINING FOR A LoRA-ADAPTED TARGET

Initialization. When LoRA fine-tuning (Hu et al., 2022) changes the target, its feature dimensions remain the same. We therefore initialize each map to the identity, $W_i = I$, so inserting the maps does not disrupt the existing drafter’s behavior with the adapted target. At initialization, $F_i W_i h_{i,t} = F_i h_{i,t}$ for every input, so the maps leave the original computation unchanged. This preserves the starting behavior with the active LoRA target, not the behavior of the target before fine-tuning. Training then learns changes $\Delta_i = W_i - I$ that help the frozen drafter predict the adapted target’s tokens.

Training data. We generate responses with the adapted target and save their tokens and hidden states. At sampled response positions, the drafter receives one known token followed by 15 masked positions to predict. Together they form a block of 16 tokens, matching the pretrained Qwen DFlash configuration (Chen et al., 2026). We retain this block structure to adapt the existing drafter without changing its prediction procedure. Each block can use target features only from before its starting position. Following DFlash, attention connects positions within a block but not separate blocks (Chen et al., 2026).

Objective. Because verification stops at the first rejected proposal, we supervise the correct prefix and the first incorrect prediction, excluding later positions. This is the accept-until-fail (AUF) objective (Yang & Li, 2026). For block b and proposal position j , let q_{bj} be the predicted token distribution, y_{bj} the saved target token, and m_{bj} a mask that excludes padding and non-response positions. We compute

$$u_{bj} = \mathbf{1} \left[\arg \max_v q_{bj}(v) = y_{bj} \text{ or } m_{bj} = 0 \right], \quad s_{bj} = \text{stopgrad} \left(\prod_{k < j} u_{bk} \right),$$

$$\mathcal{L}_{\text{AUF}} = - \frac{\sum_{b,j} m_{bj} s_{bj} \log q_{bj}(y_{bj})}{\sum_{b,j} m_{bj} s_{bj}}. \quad (5)$$

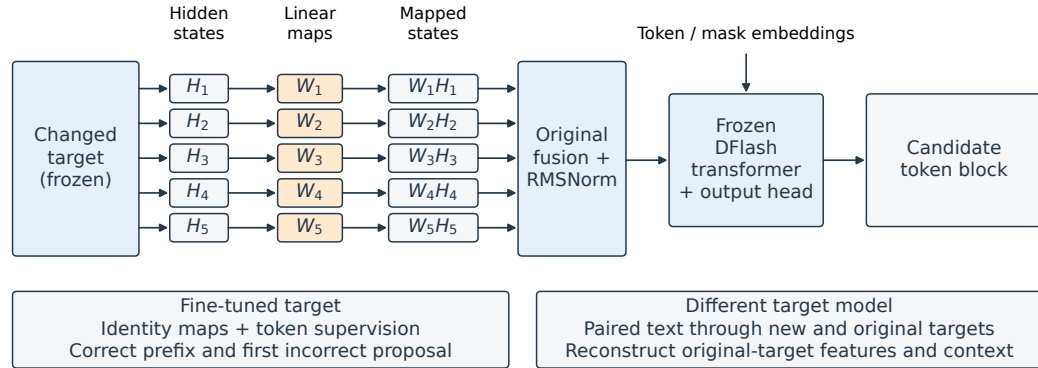
The product includes only earlier predictions, so the first failure still receives supervision. We recompute this mask from current predictions and hold it fixed during differentiation. Gradients pass through the frozen drafter to update the maps. We normalize the loss within each microbatch before averaging across accumulation steps and workers.

4.3 TRAINING FOR TRANSFER ACROSS MODELS

Initialization. A drafter trained with one target expects that target’s hidden states. Replacing the target can change both their dimensions and meaning. We therefore train the maps to reconstruct features from the original target, the model used to train the drafter. For this setting, we initialize the maps with independent Xavier-uniform values (Glorot & Bengio, 2010).

Training data. We generate responses with the new target, then run both targets causally on the same saved prompt and response to collect paired hidden states. The original target does not generate a separate response. With a shared tokenizer, both receive identical token IDs. When tokenizers

A Training: only the maps change



B Inference: maps are folded into fusion

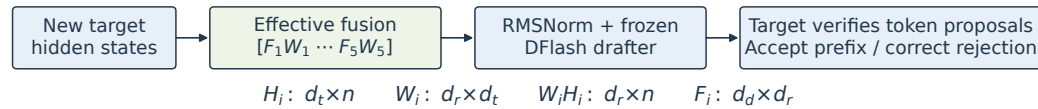


Figure 2: RelaySpec training and inference. Orange maps are trainable and blue components are frozen. Each $H_i \in \mathbb{R}^{d_t \times n}$ holds n context positions. For Qwen3-8B with the Qwen3-4B drafter, $d_t = 4096$ and $d_r = d_d = 2560$: maps are 2560×4096 and the combined projection is 2560×20480 .

differ, we pair only positions corresponding to the same text prefix. The original target is needed to prepare these training features, but is not used during adapted inference.

Objective. For example b and position t , let h_{bti} and $h_{bti}^{(o)}$ denote the paired new- and original-target features in stream i . The original features supply the context we aim to reconstruct:

$$c_{bt}^{(o)} = N \left(\sum_{i=1}^K F_i h_{bti}^{(o)} \right). \quad (6)$$

For B examples, let S_b contain the selected valid prompt and response positions in example b . Using relative squared error, we minimize

$$D(u, v) = \frac{\|u - v\|_2^2}{\|v\|_2^2 + \varepsilon}, \quad \varepsilon = 10^{-6}, \quad (7)$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{|S_b|} \sum_{t \in S_b} \left[\frac{1}{K} \sum_{i=1}^K D(W_i h_{bti}, h_{bti}^{(o)}) + D(\hat{c}_{bt}, c_{bt}^{(o)}) \right]. \quad (8)$$

The first term averages reconstruction errors across the K feature streams. The second reconstructs the combined context received by the draft layers. These two terms have equal weight. We exclude padding and average positions within each example before averaging examples. Unlike AUF, this objective trains the maps without passing gradients through the draft transformer. Dividing by each original vector’s squared norm makes the error relative to its magnitude. The context term complements the layer terms because errors in different streams can interact when the fusion projection combines them. Reconstruction is a training objective, not a guarantee of accepted tokens, so we evaluate live decoding separately.

4.4 EXTENSION TO EAGLE-3

The construction is not specific to DFlash. EAGLE-3 combines features from three target layers before autoregressive drafting (Li et al., 2025), so $K = 3$. We place one map before each of those inputs and leave the original feature projection, prediction network, and autoregressive drafting procedure unchanged. The maps fold into that projection exactly as in Equation 4, so inference again runs the original architecture with one updated projection.

We pair each setting with the same objective we use for DFlash. For a LoRA-adapted target the feature widths match, so the maps start at the identity and receive token supervision. Because EAGLE-3 drafts one token at a time instead of filling a masked block, we apply the accept-until-fail rule along its own recurrent path. At each sampled anchor the drafter predicts up to eight successive tokens, a position is supervised only when every earlier prediction was correct, and the first incorrect prediction is still supervised. A saved token continues a path only where it matches the greedy prediction. The frozen drafter carries a reduced vocabulary head, so a target token outside that head ends a path without contributing a loss term, and we normalize each example by its own count of supervised positions. For a replaced target the widths can differ, so the maps start at Xavier-uniform values and train by reconstruction using Equation 8.

4.5 GENERATION AND VERIFICATION

The new target computes hidden states during prompt processing and proposal verification. We retain states from the committed prefix and map them into context for subsequent drafting. The frozen drafter proposes tokens, and the target verifies them using its existing decoding procedure. After verification, the decoder removes rejected speculative positions from the caches before the next drafting step. The original target is no longer needed to supply features during generation. Cross-tokenizer transfer additionally requires a proposal-conversion procedure, which the hidden-state maps do not provide.

Greedy verification. For greedy decoding, changing the drafter does not change the target’s output if block verification makes the same choices as sequential target evaluation on identical prefixes. To see this, assume that the committed prefix matches autoregressive decoding. Each accepted proposal equals the target’s next choice, and the first rejected proposal is replaced by that choice.

If all proposals are accepted, the target supplies the next token. The extended prefix therefore still matches, and induction establishes the result for the complete sequence (Leviathan et al., 2023; Chen et al., 2023). This argument assumes consistent token handling, stopping rules, and target evaluation.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Drafters. Every DFlash experiment starts from one pretrained checkpoint released for Qwen3-4B (Chen et al., 2026; Yang et al., 2025). It reads five target streams at zero-based layers 1, 9, 17, 25 and 33, has draft width $2,560 \times 2,560$ fusion blocks, and drafts a 16-token block from one anchor and 15 masked positions. Section 5.4 additionally uses published EAGLE-3 checkpoints, which read three streams (Li et al., 2025). No draft transformer is ever trained.

Targets. In the *fine-tuned* setting the target is a base model carrying a LoRA adapter that the released drafter has never seen (Hu et al., 2022): Qwen3-4B with a GSM8K, KiCad or NanoCoder adapter, and Qwen3-8B with a GSM8K or CodeAlpaca (Chaudhary, 2023) adapter, each merged into the frozen target before inference. In the *replaced* setting the target is a different model from the one the drafter was built for. We evaluate four such transfers, named by the target they accelerate and the model the drafter came from: Qwen3-8B from Qwen3-4B, Qwen3-4B from Qwen3-8B, Llama3.1-8B from Qwen3-4B, and Llama3.2-3B from Llama3.1-8B (Grattafiori et al., 2024). Llama3.1-8B from Qwen3-4B is the only pair whose tokenizers differ, so it also uses the proposal bridge of Section 4.5. Maps are square 2560×2560 when the target keeps the drafter’s feature width, giving 32.8M trained parameters, and 2560×4096 when accelerating Qwen3-8B, giving 52.4M.

Reference drafters. Each setting has one meaningful reference, and we name it explicitly wherever a number is relative to it. For a fine-tuned target the reference is the *unadapted drafter*, the released checkpoint run against the fine-tuned target with no adaptation. It is the honest reference because no drafter exists for a LoRA variant, so this is what a deployment would otherwise use. For a replaced target the reference is the *native drafter*, a drafter trained specifically for the target being accelerated. This is the demanding reference, because matching it means reaching the quality of target-specific drafter training without doing any. Autoregressive decoding, the target with no drafter at all, is the common denominator for both.

Datasets and benchmarks. Transfer experiments use MATH test (Hendrycks et al., 2021), GSM8K test (Cobbe et al., 2021), pinned LiveCodeBench releases (Jain et al., 2024) and single-turn instructions derived from UltraFeedback (Cui et al., 2024), reported as Math, GSM8K, Code and Chat. Fine-tuned-target experiments evaluate on held-out prompts from the adapter’s own domain. Every reported cell uses 128 held-out prompts with a 2,048-token response cap, except KiCad at 8,192.

Metrics. RelaySpec changes only the drafter’s inputs and leaves the target and its acceptance rule untouched, so we report acceleration rather than task quality. *Speedup* is mean per-request throughput divided by that of autoregressive decoding of the same target on the same prompts, so autoregressive decoding is $1.00\times$ by definition. *Acceptance length* τ is pooled emitted tokens per verification step and includes the correction token, so its floor is one. Section 5.5 additionally reports *aggregate throughput*, total returned tokens divided by workload wall time including drain. These statistics weight requests differently and we never mix them inside one comparison. Where an ablation must choose one configuration, we select by the harmonic mean of its per-workload speedups rather than their average, because the harmonic mean of ratios is pulled down by the smallest entry and so will not let a configuration hide one weak workload behind a favourable average. We use it only for those selections.

Implementation details. For a fine-tuned target we generate 4,096 responses with that target, cache its hidden states, sample 32 anchors per example, and train one epoch, which is 512 optimizer updates at global batch 8 with AdamW at learning rate 10^{-4} . For a replaced target we reuse 16,384 saved NuminaMath (Li et al., 2024a) trajectories, supervise a fixed 25% of prompt and response positions, and train three epochs, which is 6,144 updates at learning rate 10^{-3} . Both use seed 42 on two L40S GPUs with FP32 trainable maps over a frozen BF16 model, and both export the folded projection in BF16. Single-request measurements use Transformers on one L40S with greedy

Table 1: Complete single-request results. AR is the measured autoregressive tokens per second for that target on that workload, the common $1.00\times$ baseline for every speedup in the row. Speedup is mean per-request throughput divided by AR. τ is average acceptance length including the correction token. The reference for a fine-tuned target is the released drafter used with no adaptation, so $\times\text{ref}$ above 1 is an improvement over what a deployment would otherwise run. The reference for a replaced target is a drafter trained for that target, so $\times\text{ref}$ near 1 means RelaySpec matches target-specific drafter training without performing any. A target written $A \leftarrow B$ is target A accelerated by a drafter built for B . Bold marks $\times\text{ref}$ at or above parity.

Target $\leftarrow$ drafter	Workload	AR	RelaySpec		Reference drafter		$\times$ ref
			Speedup	τ	Speedup	τ	
<i>Fine-tuned target. Reference is the released drafter, used unadapted.</i>							
Qwen3-4B + GSM8K LoRA	GSM8K	38.81	$4.63\times$	5.99	$3.32\times$	4.47	1.40
Qwen3-4B + KiCad LoRA	KiCad	38.42	$8.13\times$	11.26	$4.39\times$	5.99	1.85
Qwen3-4B + NanoCoder LoRA	NanoCoder	39.01	$4.40\times$	5.55	$3.87\times$	5.11	1.14
Qwen3-8B + GSM8K LoRA	GSM8K	37.89	$4.59\times$	6.35	$3.40\times$	4.76	1.35
Qwen3-8B + CodeAlpaca LoRA	CodeAlpaca	37.56	$4.32\times$	5.63	$4.07\times$	5.62	1.06
<i>Replaced target. Reference is a drafter trained for the target being accelerated.</i>							
Qwen3-8B $\leftarrow$ Qwen3-4B	Math	37.84	$5.82\times$	7.98	$5.81\times$	8.21	1.00
	GSM8K	37.76	$4.68\times$	6.24	$4.71\times$	6.48	0.99
	Code	37.50	$3.40\times$	4.74	$4.04\times$	5.64	0.84
	Chat	35.09	$2.31\times$	2.89	$2.55\times$	3.32	0.90
Qwen3-4B $\leftarrow$ Qwen3-8B	Math	39.42	$5.49\times$	7.66	$5.70\times$	8.09	0.96
	GSM8K	39.72	$4.27\times$	5.99	$4.46\times$	6.34	0.96
	Code	39.60	$3.30\times$	4.89	$4.34\times$	6.26	0.76
	Chat	40.48	$2.01\times$	2.83	$2.36\times$	3.36	0.85
Llama3.1-8B $\leftarrow$ Qwen3-4B	Math	41.00	$2.97\times$	4.63	$3.17\times$	5.12	0.94
	GSM8K	41.11	$2.21\times$	3.23	$3.10\times$	4.42	0.71
	Code	40.79	$1.98\times$	3.05	$3.00\times$	4.52	0.66
	Chat	40.96	$1.48\times$	2.06	$2.79\times$	3.95	0.53

non-thinking decoding, CUDA-synchronized timing and warmup excluded. Serving measurements use vLLM 0.28.0 with batch-invariant operations. In the selected single-request comparisons the unadapted, native and adapted drafters returned identical output sequences on all 128 prompts per workload, so those ratios are not confounded by output length. Under vLLM the sequences can diverge and we treat those numbers as descriptive.

5.2 MAIN RESULTS

Table 1 reports every target, workload and reference in one place. Two conclusions follow from it.

Adapting a fine-tuned target’s features recovers a large share of lost drafting quality. Training only the five maps improves every adapter at both target scales, from +6% on CodeAlpaca to +85% on KiCad, and raises acceptance length in every cell. The size of the gain tracks acceptance rather than any change to the inference path: KiCad nearly doubles acceptance length, from 5.99 to 11.26, and reaches $8.13\times$ autoregressive throughput, whereas CodeAlpaca moves from 5.62 to 5.63 and gains little. The improvement is therefore a function of how far the adapter moved the target away from what the drafter expects, which is exactly the quantity the method is designed to absorb.

A drafter can be moved to a different target model and still approach one built for it. Every transfer accelerates its target on every workload, so a frozen draft transformer can be driven by mapped features from a model it was never trained on, including across a family and a tokenizer boundary. Within the Qwen3 family the maps reach $1.002\times$ and $0.994\times$ of a native drafter on Math and GSM8K when moving up in scale, without any target-specific drafter training. Recovery is lower on Code and Chat, and lower again across families, so we report the setting as effective within a family and partial across one.

Table 2 reports a fifth transfer, a Llama3.1-8B drafter reused for Llama3.2-3B. Four trained input maps reach $2.35\times$ autoregressive throughput on Math and above $1.6\times$ on every workload.

Table 2: Llama3.1-8B drafter reused for Llama3.2-3B. Speedup is relative to autoregressive decoding (AR), the same $1.00\times$ baseline convention as Table 1.

Workload	AR (tok/s)	Speedup	τ
Math	61.90	$2.35\times$	4.78
GSM8K	64.45	$2.18\times$	4.16
Code	63.09	$1.88\times$	3.44
Chat	64.73	$1.61\times$	2.97

5.3 ABLATION STUDIES

We ablate the three decisions that define the method: where the trained parameters sit, which objective trains them, and how much capacity and supervision they need. Each ablation holds data, schedule and evaluation fixed and changes one element.

5.3.1 WHERE THE TRAINED PARAMETERS SIT

RelaySpec trains one map per feature stream and then folds those maps into the fusion block in front of which they sit. Because the folded result is a single matrix, the natural objection is that the factorization is irrelevant and any trainable matrix of the same size would do. Table 3 tests that, against four alternatives:

- **Single fused projection.** One matrix of the same total size, trained in place of the five maps and initialized to exactly the function the five maps start from. This is the decisive control, since it differs from RelaySpec only in being unfactorized.
- **Draft-transformer LoRA.** A rank-128 LoRA on the attention and MLP projections inside the draft blocks. This trains the drafter’s body instead of its inputs, and is the established alternative that RelaySpec is meant to replace.
- **Single joint map.** One dense matrix over all five concatenated streams, which permits mixing between streams at five times the parameters.
- **Rank- r maps.** Each whole map is replaced by a rank- r product, rather than adding a low-rank correction to a retained full map.

What this shows. For scale, the unadapted drafter used as the 0% reference here already reaches $3.3\times$ to $4.4\times$ autoregressive decoding before any adaptation (Table 1), so every percentage below is the further gain adaptation adds on top of that autoregressive speedup, not the whole story. The factorization does the work, not the parameter count. With identical parameters and an identical starting function, the single fused projection reaches a harmonic mean of 1.265 against 1.376 for the five maps and loses 45 points of gain on KiCad. The same contrast appears when transferring to Qwen3-8B, where under a matched objective the single fused projection reaches $0.365\times$ the native drafter on Math while the five maps reach $0.884\times$. Keeping one map per stream therefore changes what the optimizer can reach, even though the two parameterizations describe the same functions after folding and export as the same matrix. This is the behaviour that structural re-parameterization predicts, where a training-time factorization that merges away at inference still improves the trained result (Huo et al., 2026). Replacing each whole map by a low-rank product degrades monotonically, and the two smallest ranks fall below the unadapted drafter, because replacing the map discards its ability to pass the original feature through rather than merely limiting the learned correction. Draft-transformer LoRA is competitive and wins on NanoCoder, so we do not claim that adapting inputs beats adapting the body on every workload. Its cost is that the draft transformer is no longer shared, which Section 5.5 shows is what makes the method useful in deployment.

5.3.2 WHICH OBJECTIVE TRAINS THE MAPS

The two settings start from different places. With a fine-tuned target the maps start at the identity and the drafter already reads a usable interface, so what remains is to predict a shifted token distribution. With a replaced target the maps start random and possibly rectangular, so the interface itself is not yet usable. Table 4 compares the candidates and selects by harmonic mean.

Table 3: Where the trained parameters sit, for fine-tuned Qwen3-4B targets under a matched accept-until-fail objective, 4,096 examples and 32 anchors. Values are percent gain in mean throughput over the unadapted drafter. HM is the harmonic mean of the corresponding speedup ratios and is the selection criterion. Best in each column in bold.

Trained object	Parameters	GSM8K	KiCad	NanoCoder	HM
Five per-stream maps (RelaySpec)	32.8M	+ 37.6	+89.5	+8.1	1.376
Single fused projection	32.8M	+32.4	+44.5	+8.2	1.265
Draft-transformer LoRA, rank 128	36.7M	+36.4	+75.5	+ 9.8	1.355
Single joint map over all streams	163.8M	+24.3	+ 88.4	-7.0	1.244
Rank-128 maps	3.3M	+3.2	+58.4	-28.8	0.999
Rank-56 maps	1.4M	-21.9	+35.1	-49.3	0.751
Rank-28 maps	0.7M	-43.1	+12.8	-62.5	0.565

Table 4: Which objective trains the maps. The upper block reports throughput relative to the unadapted drafter for fine-tuned Qwen3-4B targets. The lower block reports throughput relative to the native Qwen3-8B drafter for transfer from Qwen3-4B, on the matched 8,192-example fits. MSE50 and MSE100 supervise that fraction of prompt and response positions. Best in each column in bold. HM is the selection criterion.

Setting	Objective	Workload				HM
		GSM8K	KiCad	NanoCoder		
Fine-tuned	Accept-until-fail	1.376	1.895	1.081		1.376
	Decaying cross-entropy	1.368	1.935	1.031		1.353
	Reconstruction, MSE50	1.011	1.191	0.977		1.052
		Math	GSM8K	Code	Chat	
Replaced	Reconstruction, MSE100	0.998	0.990	0.828	0.891	0.921
	Reconstruction, MSE50	0.999	0.975	0.832	0.888	0.919
	Decaying cross-entropy	0.886	0.919	0.406	0.573	0.623
	Accept-until-fail	0.884	0.911	0.385	0.558	0.604

What this shows. The harmonic mean makes the selection unambiguous and shows why an average would not. For a fine-tuned target the two token objectives are nearly tied on the arithmetic mean, 1.451 for accept-until-fail against 1.445 for decaying cross-entropy, because the latter’s strong KiCad result offsets its weak NanoCoder result. The harmonic mean separates them, 1.376 against 1.353, because it refuses to let the 1.031 NanoCoder entry be averaged away, and a deployment meets that workload. Reconstruction on the same fine-tuned targets reaches only 1.052, behind both token objectives and, on NanoCoder, behind the unadapted drafter itself, because it trains the maps toward the base model’s features rather than toward the tokens the adapted target actually produces. We therefore select accept-until-fail for fine-tuned targets. For a replaced target the ordering reverses and becomes decisive: both reconstruction variants exceed 0.918 while the best token objective reaches only 0.623, and the gap comes almost entirely from Code and Chat, where token supervision collapses to 0.406 and 0.573. We select reconstruction there. Among the two token objectives the harmonic mean prefers decaying cross-entropy for a replaced target, the reverse of the fine-tuned ordering, so neither token objective dominates across settings and the pairing of objective to setting is doing real work in both directions. Doubling supervised positions from MSE50 to MSE100 moves the harmonic mean by 0.002 while tripling the fitting time, so the deployed recipe supervises a quarter of positions. One qualification: the reconstruction runs use learning rate 10^{-3} and the token runs 10^{-4} , so these compare complete recipes rather than losses at one fixed setting.

Two further checks support this choice without changing it. First, on a separate matched architecture (a single low-rank fusion update rather than the five per-stream maps, held fixed to isolate the loss), plain uniform cross-entropy, the DFlash decaying weighting, and accept-until-fail were compared under identical data and schedule on NanoCoder: accept-until-fail reaches $1.072\times$ autoregressive decoding against $1.046\times$ for uniform cross-entropy and $1.047\times$ for decaying cross-entropy, a 2.4% gain over uniform weighting that decaying weighting does not reproduce. This confirms that the prefix-aware accept-until-fail weighting, not merely having some position-dependent weighting, is

Table 5: Whether the maps need to be nonlinear. Values are mean tokens per second and acceptance length under matched data, objective and schedule. The nonlinear alternative carries 16 to 20 percent more parameters and remains an active computation at inference. Best in each row in bold.

Setting	Workload	Tokens/s			τ	
		Linear	Nonlinear	Change	Linear	Nonlinear
Fine-tuned	GSM8K	179.83	177.94	−1.05%	5.988	5.953
	KiCad	312.47	321.76	+2.97%	11.256	11.411
	NanoCoder	171.73	169.74	−1.16%	5.552	5.520
Replaced	Math	220.26	215.71	−2.07%	7.984	7.980
	GSM8K	176.87	174.56	−1.31%	6.242	6.257
	Code	127.61	128.41	+0.63%	4.739	4.782
	Chat	80.93	80.78	−0.20%	2.887	2.902

what improves throughput. Second, for a fine-tuned target the training responses do not need to come from the fine-tuned target itself: fitting the same architecture from the base model’s own generations reaches $1.001\times$ the throughput of fitting from the fine-tuned target’s generations on the fine-tuned target’s evaluation prompts, with a paired 95% interval of $[0.995, 1.007]$ that contains 1. The gain therefore reflects adaptation to the workload distribution rather than a fit to the fine-tuned target’s specific outputs, which is why we generate training responses with whichever target will be evaluated rather than requiring matched labeled data.

5.3.3 WHETHER THE MAPS NEED TO BE NONLINEAR

The maps are linear because a linear map changes the feature width and still folds into the fusion projection. To test whether that costs accuracy, we add a parallel nonlinear branch to each map, so stream i contributes $W_i h_i + B_i \text{SiLU}(A_i h_i)$ (Elfwing et al., 2018), with A_i reducing to 256 coordinates and B_i initialized to zero. Training therefore starts at exactly the linear solution and departs from it only if that lowers the loss. The branch raises trained parameters from 32.8M to 39.3M for a fine-tuned target and from 52.4M to 60.9M for transfer, and unlike the linear map it cannot be folded away before inference. Table 5 holds data, objective and schedule fixed within each setting.

What this shows. The nonlinear branch is given every advantage, since it starts from the linear solution, carries more parameters and is selected by the optimizer only if it lowers the loss. It nevertheless changes throughput by between -2.07% and $+2.97\%$, leaves acceptance length almost unchanged, and is behind the linear map on five of the seven workloads. The linear form is therefore not a concession made for foldability. It is at least as accurate here and additionally collapses to one fixed matrix before inference. With differences of this size and one timed pass per cell we claim only that this nonlinear interface does not pay for itself.

5.3.4 HOW MUCH SUPERVISION THE MAPS NEED

Both recipes use small budgets, which raises three questions: how little data the maps actually need, whether the larger budgets are saturated, and whether draft-transformer LoRA would overtake the maps given more data. Figure 3 varies the supervision axes for both settings from 16 to 16,384 examples, spanning three orders of magnitude.

What this shows. Adaptation is effective from as few as 16 examples. On a fine-tuned target with only 16 examples the maps already recover part of the gap, and throughput then rises smoothly and without saturating out to 16,384 examples: GSM8K goes from $+10.8\%$ to $+45.0\%$, KiCad from $+5.0\%$ to $+113.0\%$, and NanoCoder from $+0.5\%$ to $+13.1\%$. Recovery of the native drafter under transfer follows the same pattern, rising from 15.6% to 99.9% on Math, 19.4% to 98.5% on GSM8K, 18.9% to 84.8% on Code, and 32.5% to 90.5% on Chat, so the workloads that transfer least well at large budgets are also the ones that still have the most to gain from more data. Because neither curve has saturated at the largest budget measured, the main results in Table 1 are conservative operating points rather than ceilings. Panel (b) shows the two curves crossing on the anchor axis: at the smallest anchor budget the maps lead draft-transformer LoRA clearly on KiCad, $+59.7\%$ against

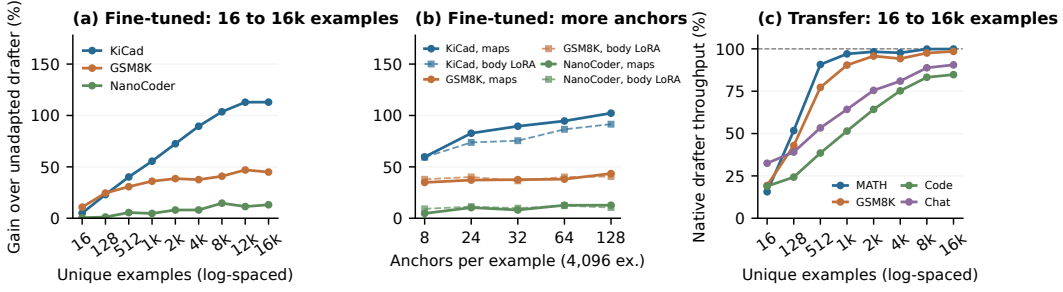


Figure 3: Supervision budget in both settings, from 16 to 16,384 training examples. (a) and (c) share a log-spaced example axis; (b) fixes examples at 4,096 and varies anchors per example. (a) and (b) use fine-tuned Qwen3-4B targets with the accept-until-fail objective, where solid lines train the five interface maps and dashed lines train a rank-128 LoRA inside the draft transformer. (c) transfers the Qwen3-4B drafter to Qwen3-8B with the reconstruction objective and plots throughput as a percentage of the native Qwen3-8B drafter. Adaptation is already effective at 16 examples and speedup keeps improving with supervision without saturating at the largest budget measured.

Table 6: Generalization to EAGLE-3, using three input maps. Token supervision is used for a fine-tuned target and reconstruction for a replaced one, following the selection in Section 5.3.2. The reference is the unadapted EAGLE-3 drafter for the fine-tuned block and a published EAGLE-3 drafter native to Qwen3-8B for the replaced block. Best in each row in bold. Acceptance uses EAGLE-3’s upstream counters and is not comparable with the DFlash tables.

Setting	Workload	Tokens/s			τ	
		Reference	RelaySpec	$\times$ ref	Reference	RelaySpec
Fine-tuned Qwen3-4B, token supervision	GSM8K	71.19	120.35	1.69	3.347	5.695
	NanoCoder	81.77	119.30	1.46	3.852	5.668
	KiCad	73.82	151.86	2.06	3.794	7.363
Qwen3-8B from Qwen3-4B, reconstruction	Math	72.18	77.99	1.08	3.627	3.638
	GSM8K	75.58	80.59	1.07	3.713	3.840
	Code	63.88	65.53	1.03	3.351	3.293
	Chat	61.82	63.41	1.03	3.186	3.136

+75.5% in the opposite direction, and at 128 anchors the two are within a few points of each other on every domain. Interface adaptation is therefore the more data-efficient choice at the low end of the supervision budget, and its advantage is largest in exactly the regime the method targets, where a new target variant must be supported from very little data.

5.4 GENERALIZATION TO A SECOND DRAFTER FAMILY

The construction should not depend on DFlash. We repeat both settings on published EAGLE-3 checkpoints, which read three target streams and draft one token at a time rather than filling a masked block, pairing each setting with the objective that Section 5.3.2 selected for it. Table 6 reports the result.

What this shows. Adapting the inputs of a frozen EAGLE-3 drafter improves it in both settings, by 45.9% to 105.7% on fine-tuned targets and above a purpose-built Qwen3-8B EAGLE-3 drafter on all four transfer workloads. Against autoregressive decoding of the same fine-tuned target, RelaySpec reaches $3.82\times$ on GSM8K, $3.75\times$ on NanoCoder and $5.00\times$ on KiCad, against $2.26\times$, $2.57\times$ and $2.43\times$ for the unadapted EAGLE-3 drafter. The objective selection also replicates, which is the stronger result. Training the same three maps by reconstruction on the fine-tuned targets, so that they reproduce the base model’s features instead of predicting the adapted target’s tokens, gives 71.89, 75.34 and 78.70 tokens per second, a 7.9% regression on NanoCoder and almost no change on GSM8K. Token supervision instead gives 120.35, 119.30 and 151.86 on the same targets and prompts, between 58% and 93% faster. Pairing the objective to the setting is therefore not a DFlash

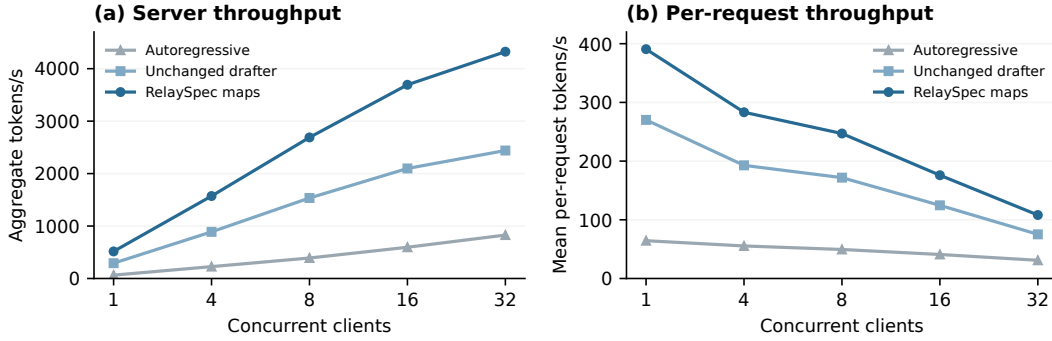


Figure 4: One Qwen3-4B target backbone and one draft backbone serving three LoRA targets under vLLM on a single L40S, 384 requests with frozen route assignments. (a) aggregate throughput, total returned tokens divided by workload wall time including drain. (b) mean per-request throughput. The adapted drafter leads on server throughput at every concurrency, while per-request throughput falls with load for all three methods.

artifact: it holds on a different draft architecture with a different drafting procedure. Two qualifications apply. Our recurrent token objective adapts the accept-until-fail rule to a sequential drafter and is not the masked-block implementation used for DFlash, so this transfers the rule rather than the code. And unlike the DFlash comparisons the EAGLE-3 runs do not return identical sequences, so we report throughput and acceptance rather than summed latency.

5.5 SERVING MANY TARGETS FROM ONE DRAFTER

Because the draft transformer is never trained, several adapted targets can share one copy of it, with only a small matrix differing between them. This is what draft-transformer LoRA gives up. Figure 4 measures the deployment on a vLLM server holding one target backbone and one draft backbone with three LoRA targets and frozen route assignments.

What this shows. The single-request gains survive batching. Aggregate server throughput is higher for the adapted drafter at every concurrency measured, reaching 4,324 tokens per second against 2,440 for the unadapted drafter and 828 for autoregressive decoding at 32 clients, while acceptance length stays near 10.5 against 5.8. Panel (b) shows why both conventions must be reported: mean per-request throughput falls with load for all three methods even as server throughput rises, so a per-request number cannot stand in for a serving number. One pooled set of maps trained on the union of three domains reaches 98.9% to 99.6% of three separately trained sets across two repeats, so a deployment need not even hold one adapter per target. This is the practical form of the reuse question that motivates target-independent drafting (An et al., 2026b). The advantage is not unbounded: on a Qwen3-8B server the adapted drafter leads at 1, 4 and 8 clients but falls within 4% of the native drafter at 16 and 32 clients, because at high concurrency the bottleneck moves away from acceptance length.

Concurrent serving across transfers. The same question arises for a replaced target: does the transferred drafter still help once requests are batched under vLLM? Table 7 reports aggregate throughput at 16 concurrent clients for the three transfers with a published native drafter. Batching reproduces the single-request ordering of Table 1 rather than changing it. On the transfer with the strongest single-request recovery, Qwen3-8B accelerated by the Qwen3-4B drafter, the mapped drafter leads on Math and GSM8K and trails narrowly on Code and Chat. On the reverse and cross-family transfers, where single-request recovery was already weaker, the mapped drafter trails the native drafter under concurrency on every workload, most on the cross-family Llama3.1-8B transfer. On the Llama3.2-3B transfer of Table 2, the same server configuration reaches $2.53\times$, $3.08\times$, $1.71\times$ and $1.77\times$ autoregressive throughput on Math, GSM8K, Code and Chat at 16 clients.

Memory and GPU usage. We measured post-request allocated and reserved GPU memory on the same T1 serving cells above. Allocated memory rises from 30.5914 GiB for the native drafter to 30.9190 GiB for the mapped drafter, a constant 0.33 GiB increase regardless of workload or concurrency; reserved memory shows the same small, workload-independent gap. This matches

Table 7: Aggregate vLLM throughput at 16 concurrent clients, total returned tokens divided by workload wall time including drain. Bold marks the higher of native and mapped.

Target $\leftarrow$ drafter	Workload	Native (tok/s)	Mapped (tok/s)	Mapped/native
Qwen3-8B $\leftarrow$ Qwen3-4B	Math	2524.31	2716.26	1.076
	GSM8K	2207.43	2262.03	1.025
	Code	1748.19	1548.47	0.886
	Chat	1091.14	1065.93	0.977
Qwen3-4B $\leftarrow$ Qwen3-8B	Math	4482.42	3710.76	0.828
	GSM8K	3160.99	3078.87	0.974
	Code	2840.15	2002.79	0.705
	Chat	1888.69	1431.21	0.758
Llama3.1-8B $\leftarrow$ Qwen3-4B	Math	1894.99	1046.94	0.553
	GSM8K	1354.50	853.13	0.630
	Code	1296.83	636.97	0.491
	Chat	1440.04	524.40	0.364

Table 8: Public-runtime comparison on 128 MATH development questions, 2,048-token cap. Speedup is each system’s own end-to-end throughput divided by its own autoregressive throughput, with a paired 95% bootstrap interval. This uses the earlier single-projection interface, not the five-map DFlash construction evaluated elsewhere in this section, and different runtimes and inherited base models prevent attributing the ranking to the adaptation method alone. Bold marks the highest observed speedup.

System	End-to-end (tok/s)	Own AR (tok/s)	Speedup [95% CI]
Single-projection interface	193.68	37.72	5.14 [4.90, 5.37]
PARDo	105.63	31.67	3.34 [3.22, 3.45]
SD ² (frozen drafter)	19.92	12.48	1.60 [1.56, 1.63]

expectation: the trained maps add 52.4 million BF16 parameters to an 8-billion-parameter target and a several-billion-parameter draft transformer, so the memory cost of adaptation is negligible relative to the models it accelerates, and it does not grow with client count.

5.6 COMPARISON WITH PUBLISHED SYSTEMS

The reports also include a public-runtime comparison of an earlier single-projection version of this interface against two published drafters that reuse or steer a fixed prediction network: PARDo, which reuses a released parallel drafter with no new target-specific fitting (An et al., 2026a), and SD², evaluated in its frozen-drafter configuration, which steers a pretrained autoregressive drafter through injected features (Berdoz et al., 2026). All three ran on the same 128 MATH development questions with a 2,048-token cap, each reported against its own runtime’s autoregressive baseline because the systems differ in precision, attention backend and supporting libraries. Table 8 reports the result.

5.7 ADAPTATION COST

Reusing a drafter is only worthwhile if adapting it costs much less than training one. We cannot compare the two in GPU-hours from published numbers, because the DFlash paper reports its training data and schedule but not its training time or hardware budget, and it trains on different accelerators than we use (Chen et al., 2026). We therefore compare how much supervision each recipe consumes and report measured time only for our own fits, keeping fitting separate from data preparation as RepSpec does (Huo et al., 2026).

The published DFlash recipe trains all five draft layers on about 800,000 samples for six epochs, sampling 512 anchor positions per sequence at every epoch (Chen et al., 2026). Table 9 reports how many examples RelaySpec needs for each setting and the resulting reduction against that published recipe.

Table 9: Training examples required, against the published DFlash recipe of about 800,000 samples (Chen et al., 2026). Every fit here runs on two L40S GPUs in under half an hour.

Setting	Training examples	Reduction vs. published DFlash
Fine-tuned target (DFlash, five maps)	4,096	195× fewer
Replaced target (DFlash, five maps)	16,384	49× fewer
Fine-tuned target (EAGLE-3, three maps)	4,096	195× fewer
Replaced target (EAGLE-3, three maps)	16,384	49× fewer

What this shows. Adaptation is cheap enough that supporting a new target variant is a routine operation rather than a training project: two orders of magnitude less data than training a drafter from scratch, and every fit above completes in under half an hour on two GPUs. Reconstruction is additionally the cheaper objective per example because it sends no gradient through the draft body, so the transfer fits are faster per example than the token-supervised fits despite using four times as many examples. Preparation is the honest counterweight and is not zero: reconstruction needs features from both the new and the original target on the same tokens, while token supervision needs features from the adapted target only, and generating the training responses is a further reusable cost we do not charge per fit. The comparison against published drafter training therefore holds on supervision consumed, and we do not extend it to an end-to-end wall-clock ratio.

6 ANALYSIS

7 CONCLUSION

AI USE STATEMENT

ETHICS STATEMENT

REPRODUCIBILITY STATEMENT

REFERENCES

- Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost parallel draft model adaptation. In *International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=XbOyv7iVGL>.
- Zihao An, Taichi Liu, Ziqiong Liu, Dong Li, Ruofeng Liu, and Emad Barsoum. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding. *arXiv preprint arXiv:2605.08632*, 2026b. URL <https://arxiv.org/abs/2605.08632>.
- Frédéric Berdoz, Peer Rheinboldt, and Roger Wattenhofer. Steering pretrained drafters during speculative decoding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://ojs.aaai.org/index.php/AAAI/article/view/40255>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 5209–5235. PMLR, 2024. URL <https://proceedings.mlr.press/v235/cai24b.html>.
- Sahil Chaudhary. Code Alpaca: An instruction-following LLaMA model for code generation. GitHub repository, <https://github.com/sahil280114/codealpaca>, 2023.
- Alan Chen, Jack Merullo, Alessandro Stolfo, and Ellie Pavlick. Transferring linear features across language models with model stitching. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.nips.cc/paper_files/paper/2025/hash/4569a868e7aa891248832ec08445d071-Abstract-Conference.html.

- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Yunfei Cheng, Aonan Zhang, Xuanyu Zhang, Chong Wang, and Yi Wang. Recurrent drafter for fast speculative decoding in large language models. *arXiv preprint arXiv:2403.09919*, 2024. URL <https://arxiv.org/abs/2403.09919>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Bingxiang He, Wei Zhu, Yuan Ni, Guotong Xie, Ruobing Xie, Yankai Lin, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with scaled AI feedback. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2310.01377>.
- Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018. URL <https://arxiv.org/abs/1702.03118>.
- Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pp. 249–256. PMLR, 2010. URL <https://proceedings.mlr.press/v9/glorot10a.html>.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- Youcheng Huang, Chen Huang, Duanyu Feng, Wenqiang Lei, and Jiancheng Lv. Cross-model transferability among large language models on the platonic representations of concepts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3686–3704. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.185. URL <https://aclanthology.org/2025.acl-long.185/>.
- Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. Repspec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554c1f-Abstract-Conference.html.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.

- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. Hugging Face repository, <https://huggingface.co/AI-MO/NuminaMath-CoT>, 2024a.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pp. 28935–28948. PMLR, 2024b. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024c. URL <https://aclanthology.org/2024.emnlp-main.422/>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/c7b5a35ea98b62512a869c19ea7b03cb-Abstract-Conference.html.
- Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. Efficiently aligning draft models via parameter- and data-efficient adaptation. *arXiv preprint arXiv:2603.09527*, 2026. URL <https://arxiv.org/abs/2603.09527>.
- Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Alvin Cheung, Zhijie Deng, Ion Stoica, and Hao Zhang. Online speculative decoding. *arXiv preprint arXiv:2310.07177*, 2024. URL <https://arxiv.org/abs/2310.07177>.
- Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. Omnidraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. In *Advances in Neural Information Processing Systems*, volume 38, 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/3c2fe1417eed1c6ff9acf169617981ea-Paper-Conference.pdf.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Tianjian Yang and Meng Li. Spec-AUF: Accept-until-fail training under train-inference misalignment for masked block drafters. *arXiv preprint arXiv:2607.01893*, 2026. URL <https://arxiv.org/abs/2607.01893>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/1e8a19426224ca89e83cef47f1e7f53b-Abstract.html>.
- Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/8766fbc68e1ed1cdef712ce273e0a363-Abstract-Conference.html.